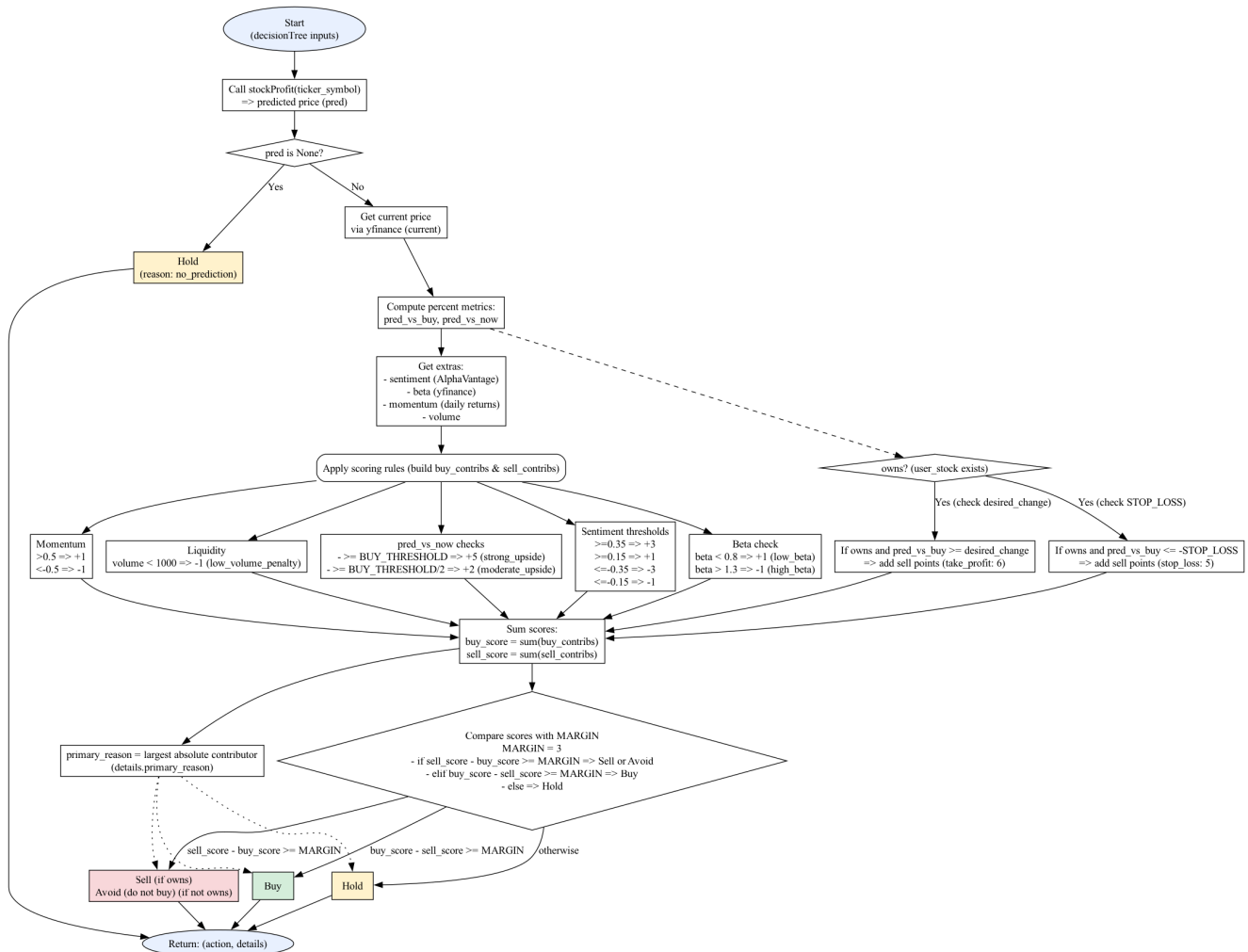


Ibis Study Documentation

GitHub Link: <https://github.com/appleshoops/ibis-study>

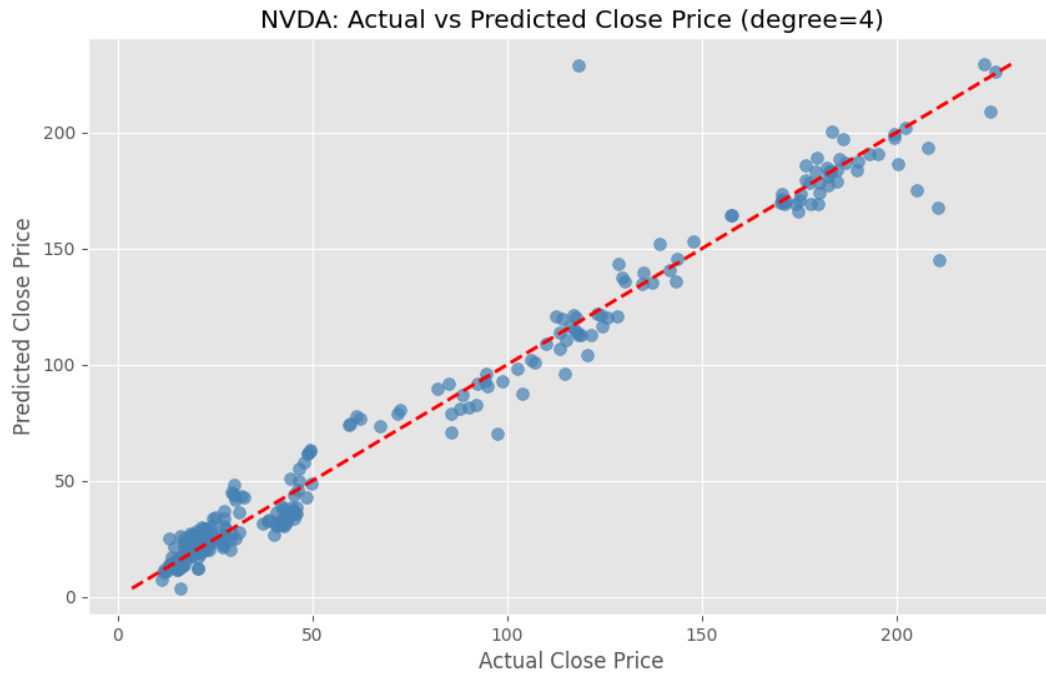
Decision Tree For Algorithm



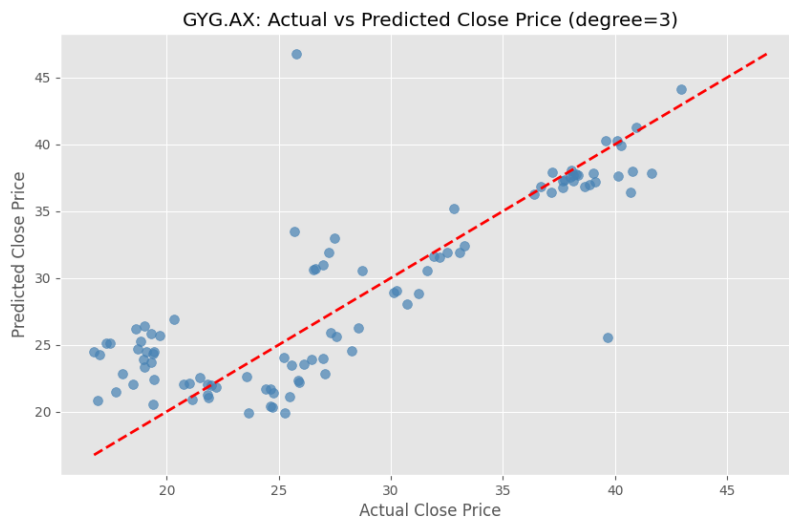
Data Dictionary

variable	description	data_type	data_format_notes
calcDailyReturns(ticker_symbol)	Fetches 5-year history and calculates daily percent change of Close	function(param: str) -> pandas.Series or None	Returns pandas.Series (index: DatetimeIndex trading days), float values such as 0.012 (1.2%). On invalid ticker the function may return None (note: current code has an inconsistent early-return tuple in invalid branch).
calcHighLowDiff(ticker_symbol)	Compute rolling 52-week high/low and position within range	function(param: str) -> pandas.DataFrame or None	Index: DatetimeIndex (daily). Columns: High, Low, Close, 52_week_high, 52_week_low, position_in_52w_range (float ~0.0-1.0). Returns ~5-year slice. NaN possible if high=low.
etfComparison(ticker_symbol)	Find top ETF for the stock's sector and return ETF history	function(param: str) -> pandas.DataFrame or None	Index: DatetimeIndex (daily). Columns include Open, High, Low, Close, Volume. Used: Close and Jlo[-1]. May be None if sector/ETF unavailable.
volumeCollect(ticker_symbol)	Return current reported volume for ticker	function(param: str) -> int/float or None	Single numeric value (e.g., 1_234_567). None on fetch error or missing key.
stockDataCollection(ticker_symbol)	Assemble features DataFrame for modeling (Close, Volume, 52-week position, ETF close, daily returns)	function(param: str) -> pandas.DataFrame or None	Index: date objects (converted via .index.date). Columns: Close(float), Volume(int), 52 Week Range Position(float), Top Sector ETF Close(float), Daily Returns(float). Rows: trading days; NaNs dropped.
import-side-effect	volumeCollect('NVDA') call on import	side-effect	function call
trainModel(ticker_symbol)	Train polynomial (degree=2) regression (Ridge by default), save artifacts, and display actual vs predicted plot	function(param: str) -> None (side-effects)	Requires training_data (pandas.DataFrame). Side-effects: creates poly/, models/, scaler/ directories and writes joblib .pkl files; shows matplotlib plot via plt.show().
training_data	Data from yfCollectData.stockDataCollection used for training	pandas.DataFrame	Columns: Close, Volume, 52 Week Range Position, Top Sector ETF Close, Daily Returns. Index: date objects.
X	Feature dataframe used for model (before split)	pandas.DataFrame	Columns: Volume, 52 Week Range Position, Top Sector ETF Close, Daily Returns. Shape (n_samples, 4).
y	Target dataframe containing Close price	pandas.DataFrame	Column: Close (float). Shape (n_samples, 1).
X_train	X_test	y_train float y_test	
weight_Volume	Manual multiplicative weight for Volume	float	0.172
weight_52_week_pos	Manual multiplicative weight for 52-week position	float	0.264
weight_etf	Manual multiplicative weight for Top Sector ETF Close	float	0.466
weight_daily_returns	Manual multiplicative weight for Daily Returns	float	0.087
X_train_w	X_test_w	Weighted feature dataframes (after applying weights)	pandas.DataFrame
scaler	StandardScaler fit on X_train_w	sklearn.preprocessing.StandardScaler	Transforms to numpy.ndarray. Saved to scaler/[ticker]_scaler_grades.pkl
X_train_scaled	X_test_scaled	Scaled feature arrays	numpy.ndarray
poly	PolynomialFeatures transformer (degree=2)	sklearn.preprocessing.PolynomialFeatures	include_bias=False. Saved to poly/[ticker]_poly_transformer_grades.pkl
X_train_poly	X_test_poly	Polynomial features arrays	numpy.ndarray
use_ridge	ridge_alpha	Model choice and regularization parameter	bool/float
model	Trained regression model (Ridge or LinearRegression)	sklearn.linear_model.Ridge or LinearRegression	Fitted model; exposes .coef_ and .predict. Saved to models/[ticker]_polynomial_regression_model_grades.pkl
feature_names	Names for polynomial features	numpy.ndarray(str)	From poly.get_feature_names_out(X.columns)
coef_df	Absolute coefficient values per polynomial term	pandas.DataFrame	Columns: Feature, Abs Coefficient. Used to compute aggregated importance per base feature.
volume_imp	week_52_pos_imp	etf_imp	daily_returns_imp
y_test_pred	Predicted test set targets	numpy.ndarray or array-like	Used to compute Test R ² and MAE; plotted vs actual.
poly_path	model_path	joblib .pkl files	Artifact file paths where transformers and model are saved
saved_artifacts	Persisted transformer, model, scaler	joblib .pkl files	poly: PolynomialFeatures; models: Ridge/LinearRegression; scaler: StandardScaler
plot_actual_vs_predicted	Matplotlib scatter plot of actual vs predicted Close	matplotlib.figure	plt.show() is called (interactive). In headless environment consider saving figure instead.
userStockBuyPrice(user_id)	ticker_symbol	Query SQLite database for user's average buy price	function(param) -> float or None
stockProfit(ticker_symbol)	(Re)train model, load artifacts, fetch current features and predict price	function(param) -> float or None	Loads poly, model, scaler via joblib. Builds single-row DataFrame of current features (Volume, 52 Week Range Position, Top Sector ETF Close, Daily Returns), applies weights then scaler-poly to predict. Returns float predict
features	list of input feature names used by model	list(str)	['Volume', '52 Week Range Position', 'Top Sector ETF Close', 'Daily Returns'] — ordering required for model input
weight_Volume	weight_52_week_pos	weight_etf	weight_daily_returns
poly	model	scaler	Loaded persisted transformer, model, scaler
volume	etf_hist	etf_current	position_in_52w_range_hist
X_current	Single-row DataFrame containing current feature values to be predicted	pandas.DataFrame	One row. Columns match features list. Values are weighted before scaling.
getNewsSentiment(ticker_symbol)	Fetch AlphaVantage NEWS_SENTIMENT and parse overall_sentiment_score	function(param: str) -> float or None	HTTP GET to AlphaVantage. Returns float score (or None). Note: API key is hard-coded in code; watch rate limits.
getStockBeta(ticker_symbol)	Fetch beta from yfinance.ticker.info	function(param: str) -> float or None	Reads stock.info.get('beta'). May be None or missing; can be slow.
decisionTree(ticker_symbol)	user_id=None	desired_change=None	Rule-based decision engine combining prediction, ownership, and auxiliary signals to output action and details
BUY_THRESHOLD	Buy threshold used for pred_vs_now checks	float	8.0 (percent)
STOP_LOSS	Stop-loss threshold used for pred_vs_buy checks	float	5.0 (percent)
MARGIN	Minimum score difference to act (Buy vs Sell)	int	3 (points)
details	Dictionary of diagnostic and signal values returned by decisionTree	dict	Contains keys: predicted_prices, user_buy_price, owns, current_price, predicted_pct_vs_buy, predicted_pct_vs_now, sentiment, beta, momentum, volume, buy_contribs, sell_contribs, buy_score, sell_score, primary_reason
predicted_price	Predicted price returned from stockProfit	float or None	Stored as details['predicted_price']
user_buy_price	User's average buy price fetched from DB	float or None	Stored as details['user_buy_price']
owns	Whether user owns the stock	bool	True if user_buy_price is not None
current_price	Current market price fetched from yfinance	float or None	From Lintz['regularMarketPrice'] or [previousClose] or 1d history Close
predicted_pct_vs_buy	predicted_pct_vs_now	Percent metrics comparing prediction to buy/current	float or None
sentiment	News sentiment score from AlphaVantage	float or None	Range depends on API; thresholds used in rules: 0.35, 0.15
beta	Stock beta value from yfinance	float or None	Used thresholds: <0.8 => low_beta (+1), >1.3 => high_beta (-1)
momentum	Short-term momentum computed from last 5 daily returns	float or None	Computed as mean(last5_returns) * 100 (percent). Thresholds used: >0.5 => +1, < -0.5 => -1
volume	Current volume used for liquidity check	int/float or None	If volume < 1000 => low_volume_penalty applied (adds a negative buy contribution of -1)
buy_contribs	sell_contribs	Lists of (points, reason) tuples collected during scoring	list of tuples
buy_score	sell_score	Summed contribution scores	numeric (int/float)
primary_reason	Contributor with largest absolute points	dict or None, {'points': <num>}	'reason': <str>''
final action	Returned action string from decisionTree	str	'Sell' if sell_score - buy_score >= MARGIN (and owns) else 'Avoid (do not buy)' if not owns; 'Buy' if buy_score - sell_score >= MARGIN; else 'Hold'
side-effects and external dependencies	Databases and artifacts used	various	database.db (Portfolio table), files in poly/, models/, scaler/ created by trainModel, AlphaVantage API (key in code), yfinance network calls

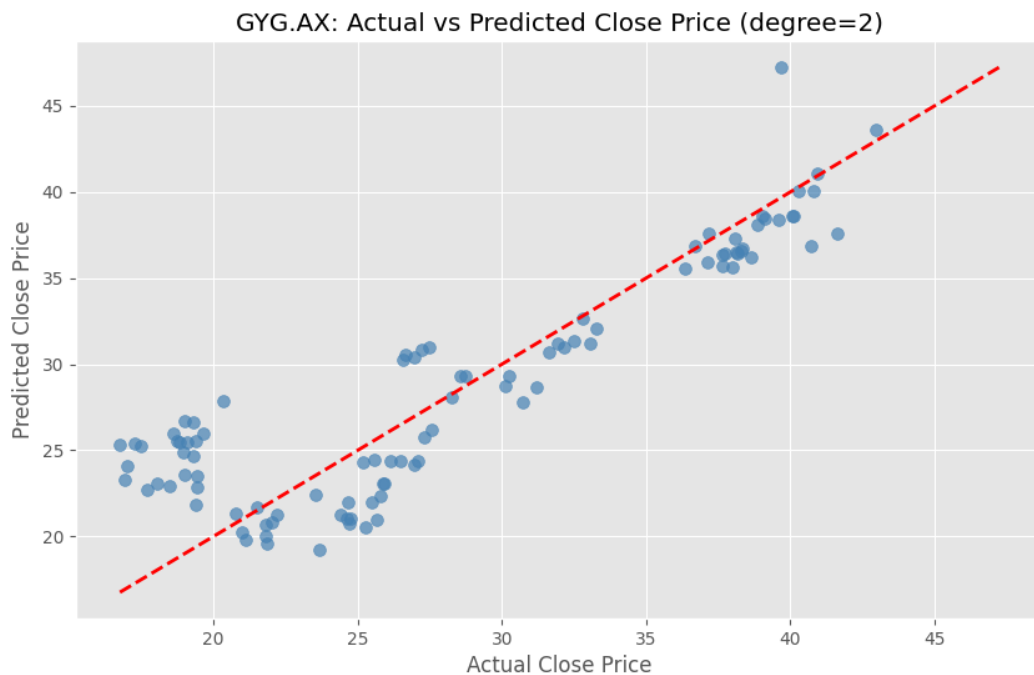
Testing



Testing NVIDIA with degree 4



Testing GYG with degree 3



Testing GYG with degree 2

```
✓ Saved polynomial transformer to poly/CYL.AX_poly_transformer_grades.pkl
✓ Saved model to models/CYL.AX_polynomial_regression_model_grades.pkl
✓ Saved scaler to scaler/CYL.AX_scaler_grades.pkl
ok model train yay
Beta for CYL.AX: 1.246
Decision for CYL.AX: Buy (Buy score: 6, Sell score: 0)
```

Testing algorithm for CYL.AX

```
$ABCD: possibly delisted; no price data found (period=5y)
```

Testing with invalid ticker, returns an Exception as try statement fails

Ethical And Social Issues

The use of machine learning in finance is extremely limited as stock markets and other financial institutions are often unpredictable and cannot be understood through just numerical patterns. They often require a good understanding of politics, current affairs, and how society works in general in order to make a profit, buying and selling at the same time. This is harder for machine learning as it learns from patterns and small scale models such as ours do not have full access to the internet and news headlines, only what we give it. Until something like AGI is achieved, it is impossible for ML to fully comprehend the nuances of the stock market.

There is also the risk of over-relying on the model. While it may be able to predict the direction and even the price of changes in the stock market, at this level, it requires strict human supervision. Investing without checking if its a good idea can lead to significant monetary losses. Over-relying on the model can make a person complacent.

When training ML and AI, there is always a risk of there being bias in the training data. While the yfinance prices are generally objective, some statistics such as stock beta may not always be objective. Additionally, the use of news headlines from various sources can be considered adding bias as the sources aren't checked for bias and could be written to portray a company positively or negatively.

This tool is also meant to only be used for educational purposes. It is not meant to be seriously used as a guide for trading and doing so could cause major financial losses at the user's fault. The training data is limited and so is the compute as it is only meant to show what is possible.

Selection and Justification of ML Algorithm

We decided to use a polynomial regression model for this project. This was done as polynomial regression allows for complex and nuanced curves to be used when training an ML model. It allows for several features to be integrated and for non-linear trends to be investigated which is essential for something as complex as the stock market which is always changing and can be unpredictable in a way that linear regression cannot help with. I used degree 2 for the polynomial as during the testing phase, it brought the predicted values closest to the actual values along the line $y=x$ and preventing overfitting the graph.